

Support Vector Machines

Dr. Krishna Bathula

Learning Objectives

1. Feature Space
2. Hyperplane
3. Support Vector Machine
4. Soft Margin
5. Outlier Detection
6. K Nearest Neighbor

Feature Space

Feature space refers to the **collections of features** used to characterize the data.

Given some data, a feature space is the **set of all possible values** for a set of features from that data.

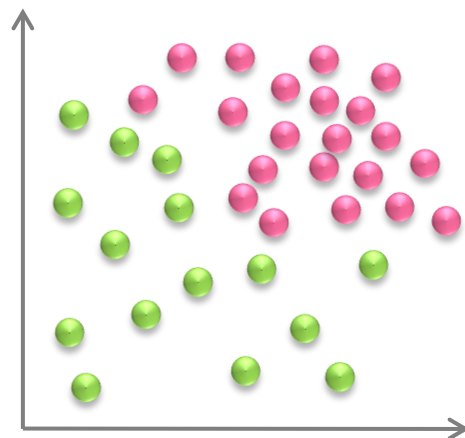
Example: If your data is about people, your feature space might consist of Gender, Height, Weight, Age...

It is always possible to represent **feature values** and thus, a feature space using only numbers.

Feature space will have **n-dimensions** based on the number of variables considered.

Feature Space - Example

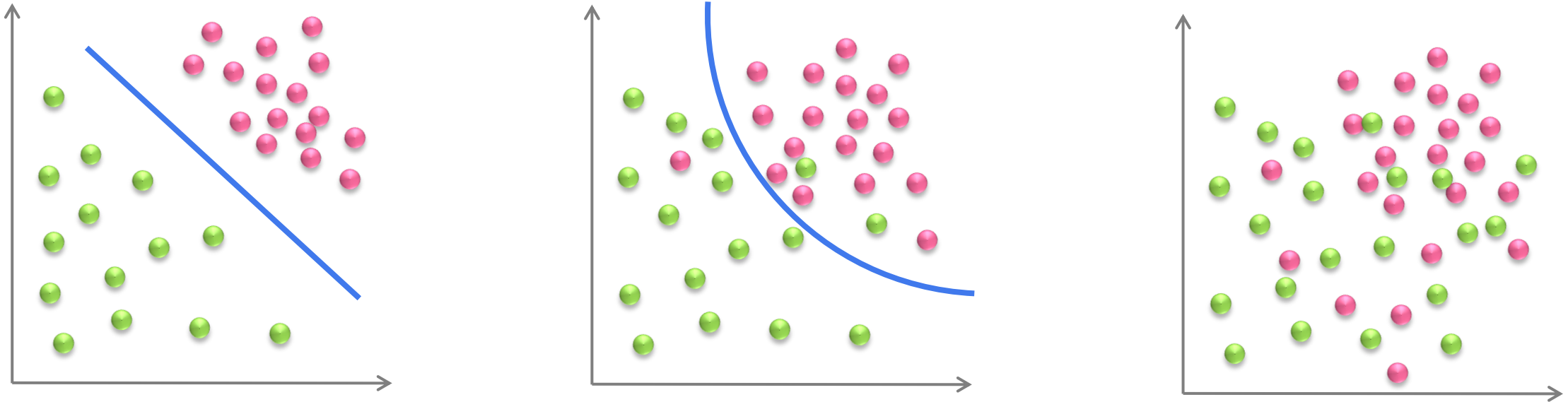
Given some labeled data on cats and dogs, we can choose some appropriate **features** and plot the data in our feature space.



When data points are close to each other in the feature space, they are likely to represent conceptually similar or related entities.

If we see a clean clustering of data-points that represent dogs in our feature space, then the features in our data must be characterizing dogs well.

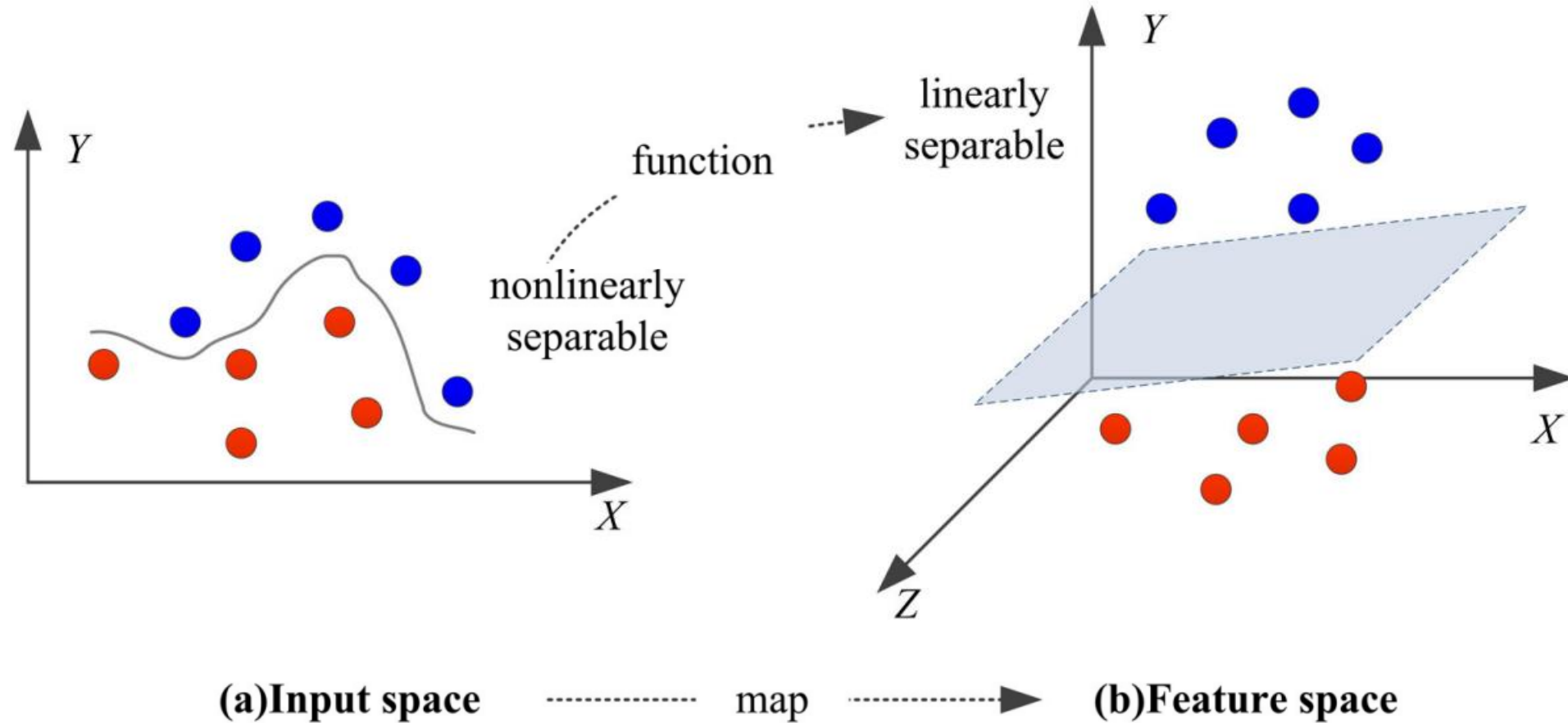
Feature Space



The first two plots have a relatively **good feature space**.

When the feature space is good, the classification problem is based on how to best **divide** up the **feature space** to capture the concept with minimal error.

Mapping Input Space to Feature Space

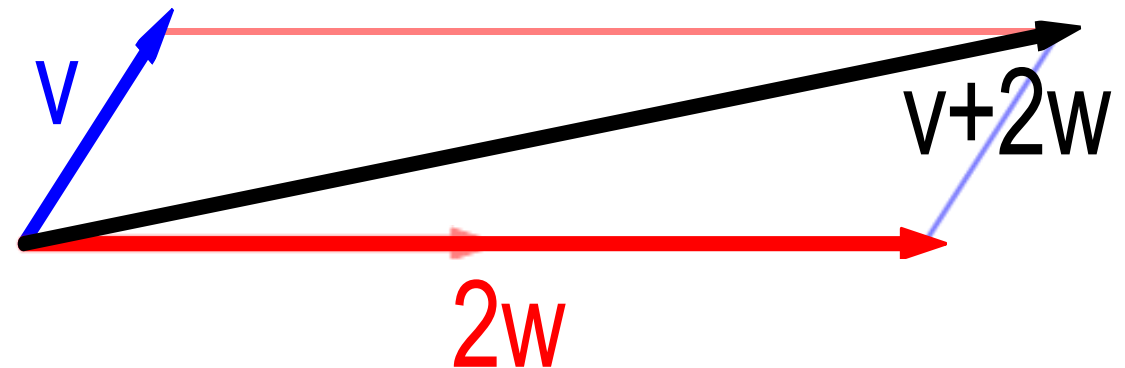
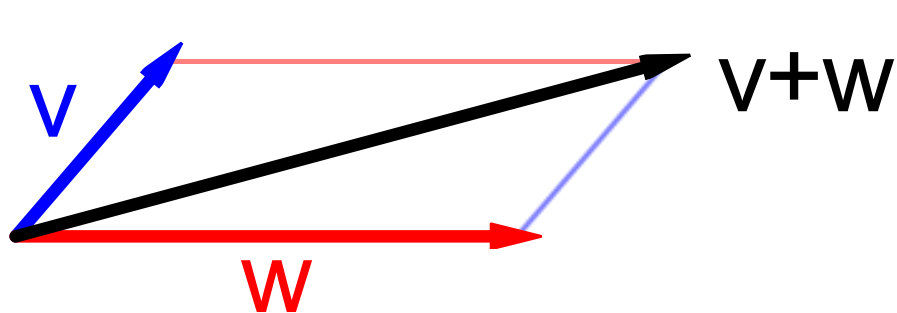


Vectors Spaces

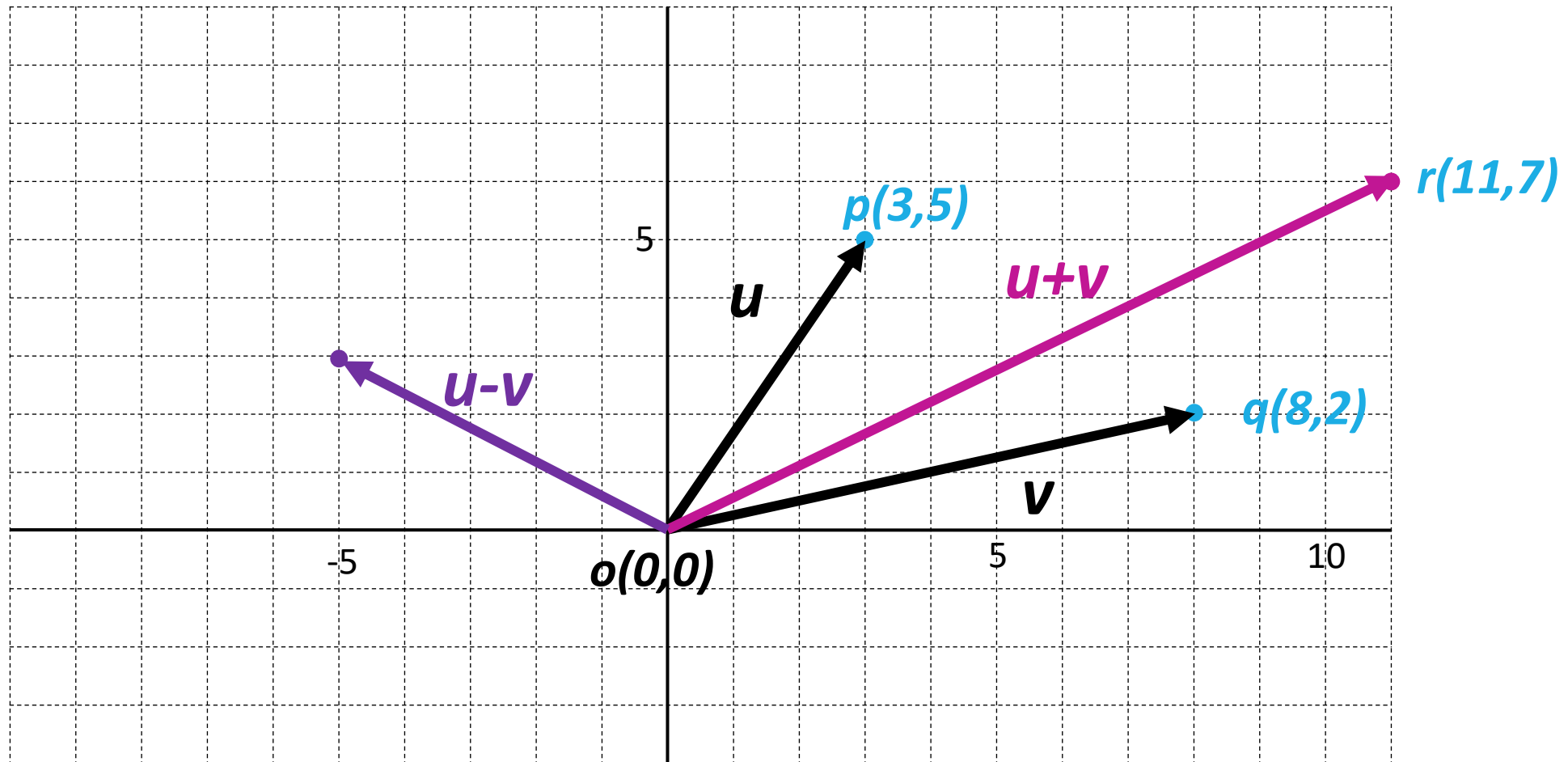
A vector space is a set whose elements are often called vectors.

1. These vectors may be added together and
2. Also Multiplied by numbers called scalars.

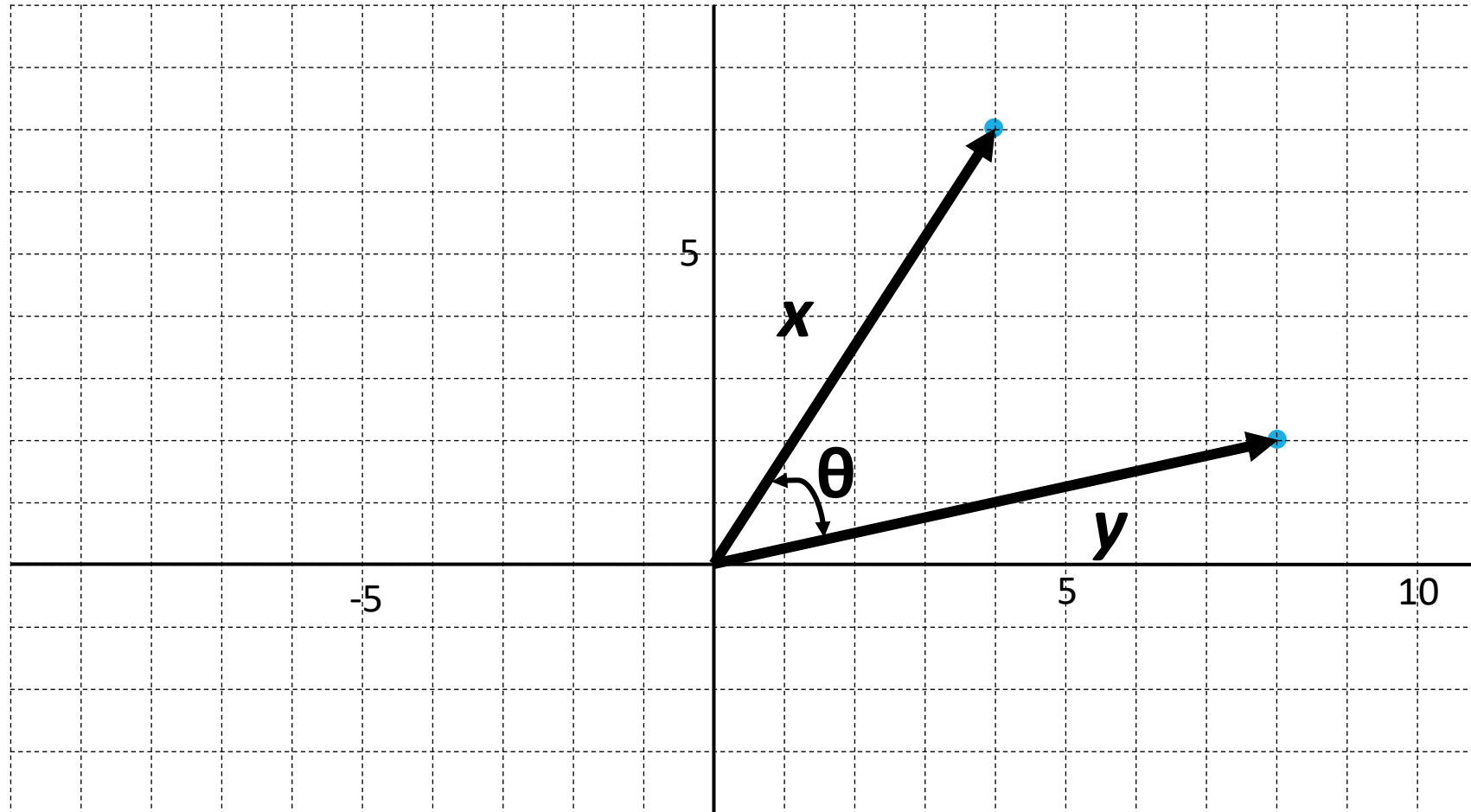
Vector spaces are characterized by their dimension, which specifies the number of independent directions in the space.



Vectors Space – Add & Subtract



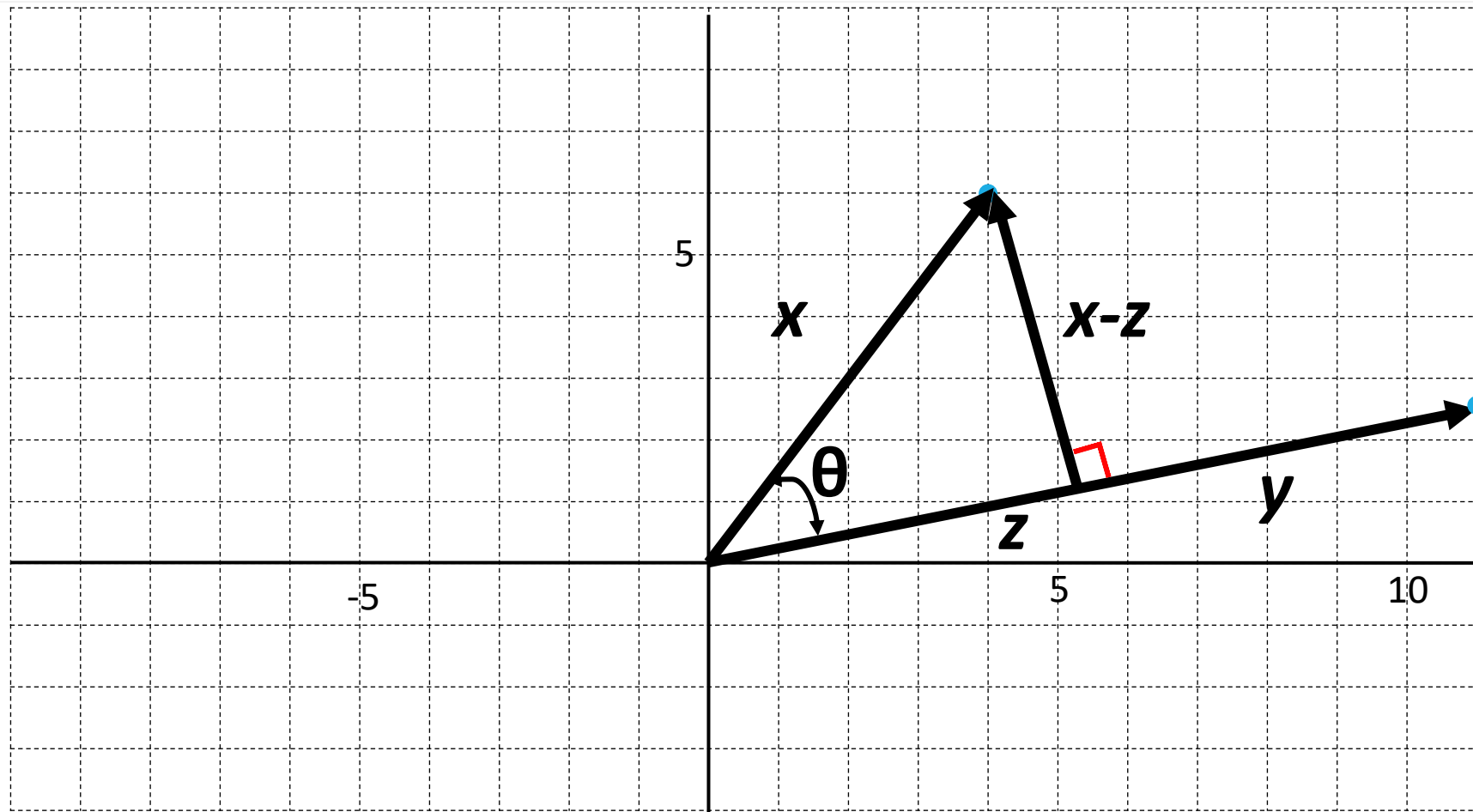
Vectors – Dot Product



$$\mathbf{x} \cdot \mathbf{y} = x_1y_1 + x_2y_2 = \sum_{i=1}^2 (x_iy_i)$$

$$\|\mathbf{x}\| \|\mathbf{y}\| \cos(\theta) = \mathbf{x} \cdot \mathbf{y}$$

Vectors – Orthogonal Projection



The **orthogonal projection** computes the distance between x and the line which goes through y ($x-z$)

Decision Boundary

In a statistical-classification problem with **two classes**, a **decision boundary** or **decision surface** is a **hypersurface** that partitions the **underlying vector space** into two sets, one for each class.

The classifier will classify all the points on **one side** of the decision boundary as belonging to **one class** and all those on the **other side** as belonging to the **other class**.

A **decision boundary** is the region of a problem space in which the output label of a classifier is ambiguous.

If the **decision surface** is a **hyperplane**, then the classification problem is linear, and the classes are linearly separable.

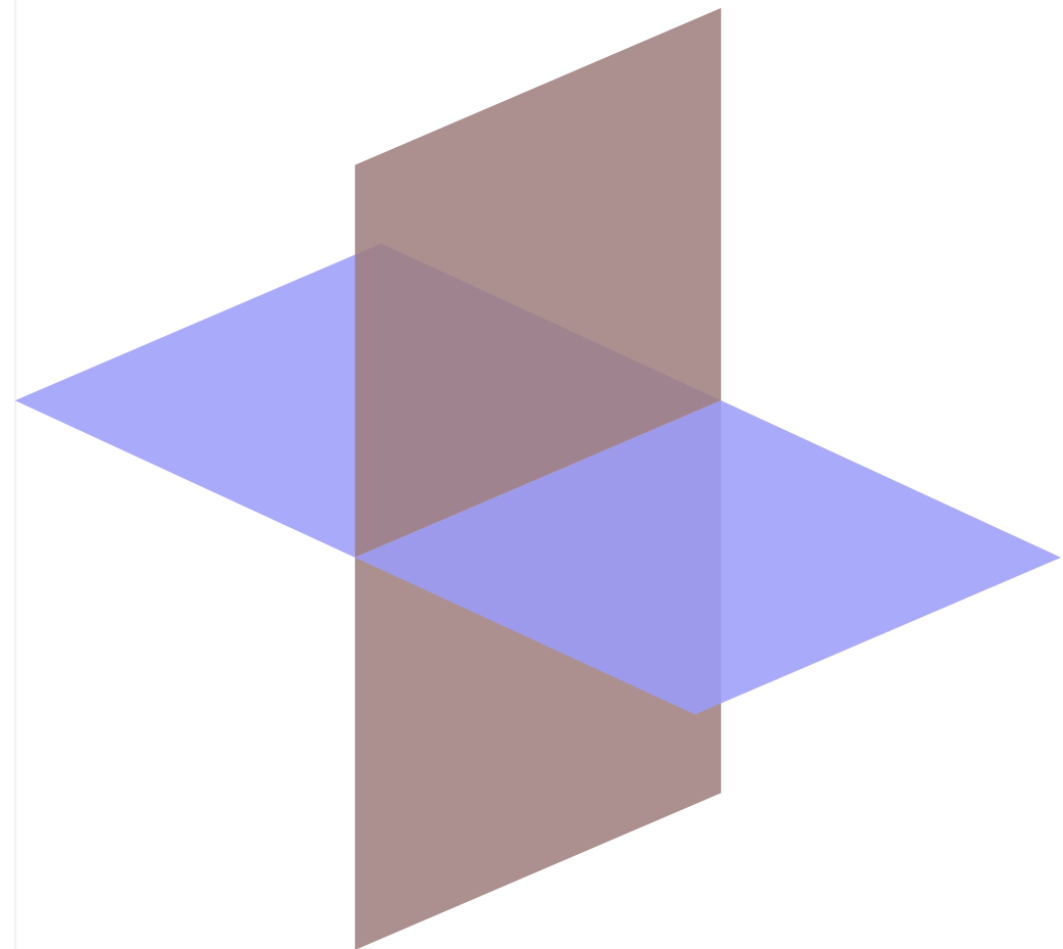
Hyperplane

In geometry, a **Hyperplane** is a subspace whose dimension is one less than that of its ambient space.

For example, if a **space** is **3-dimensional**, its **hyperplanes** are the **2-dimensional planes**.

While if the **space** is **2-dimensional**, its **hyperplanes** are the **1-dimensional** lines.

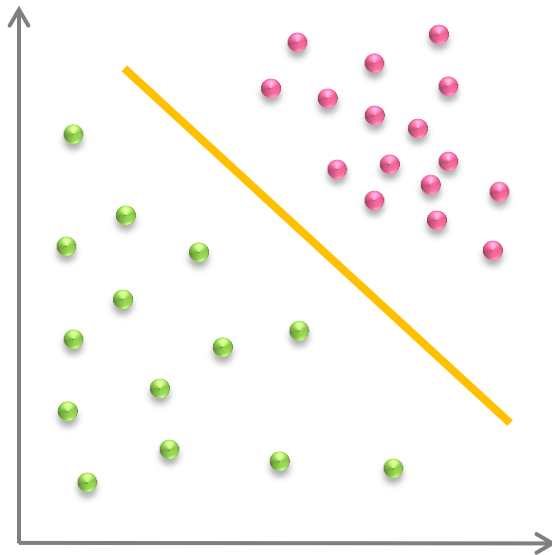
This notion can be used in any general space in which the concept of the **dimension** of a **subspace** is defined.



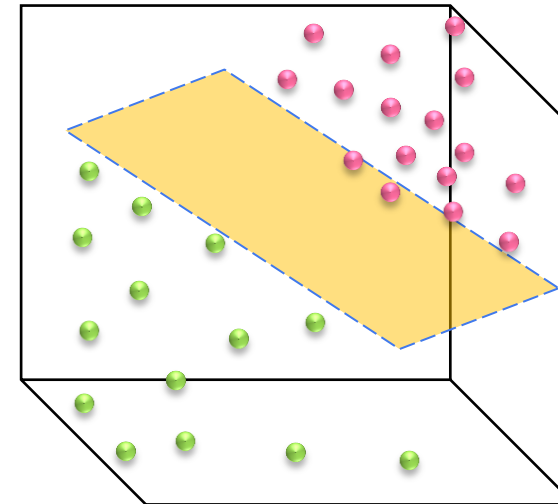
Vector hyperplanes

In a **vector space**, a vector **hyperplane** is a subspace of co-dimension 1, only possibly shifted from the origin by a vector, in which case it is referred to as a flat.

Such a hyperplane is the solution of a **single linear equation**.



Line
 $y=mx+b$



Hyperplane
 $w^T \cdot x = 0$

Hyperplane - Decision Boundaries

Hyperplanes are decision boundaries that help **classify** the **data points**.

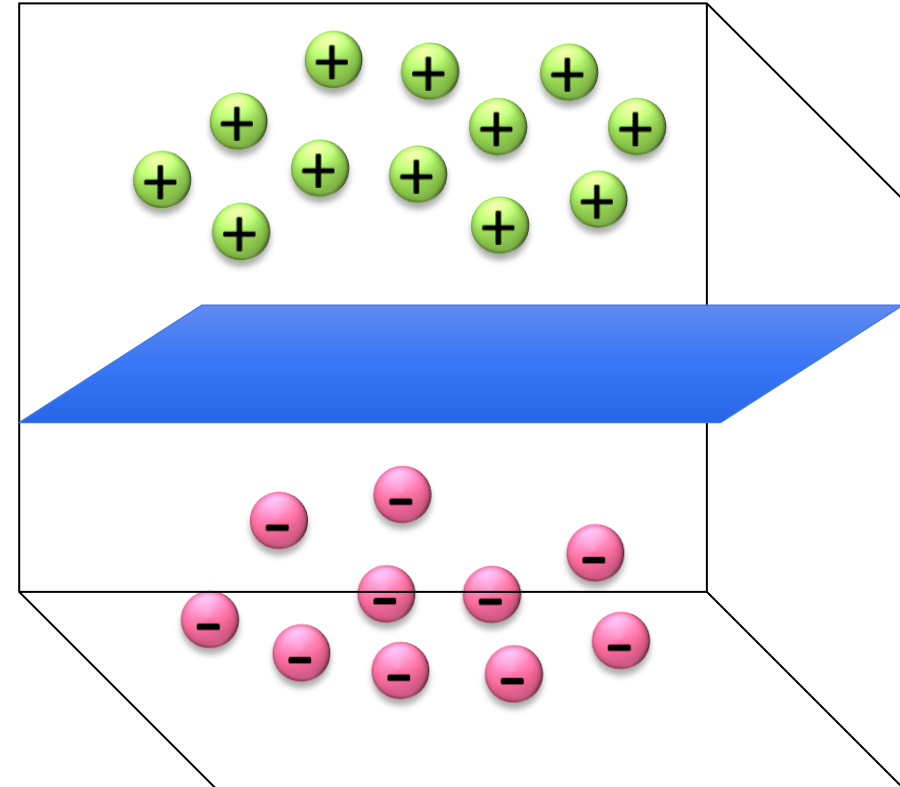
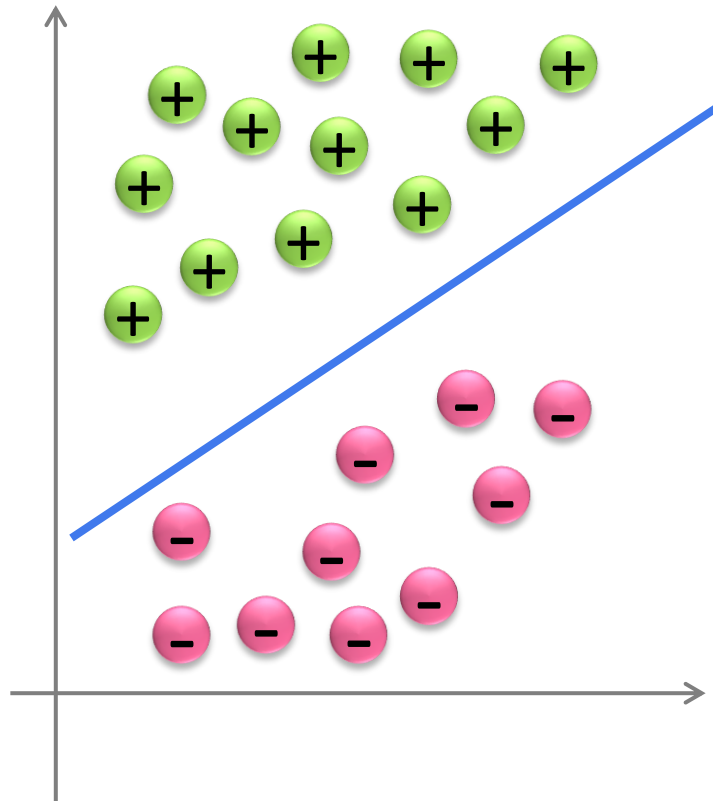
Data points falling on **either side** of the **hyperplane** can be attributed to different classes.

Also, the dimension of the hyperplane depends upon the **number of features**. If the number of input **features is 2**, then the hyperplane is just a **line**.

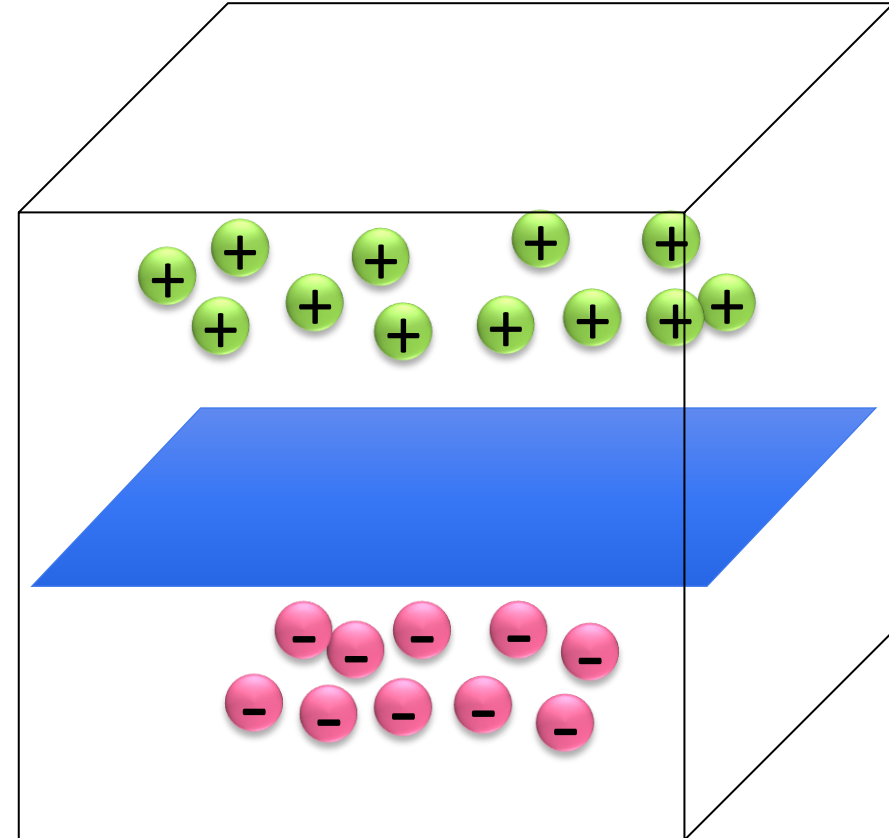
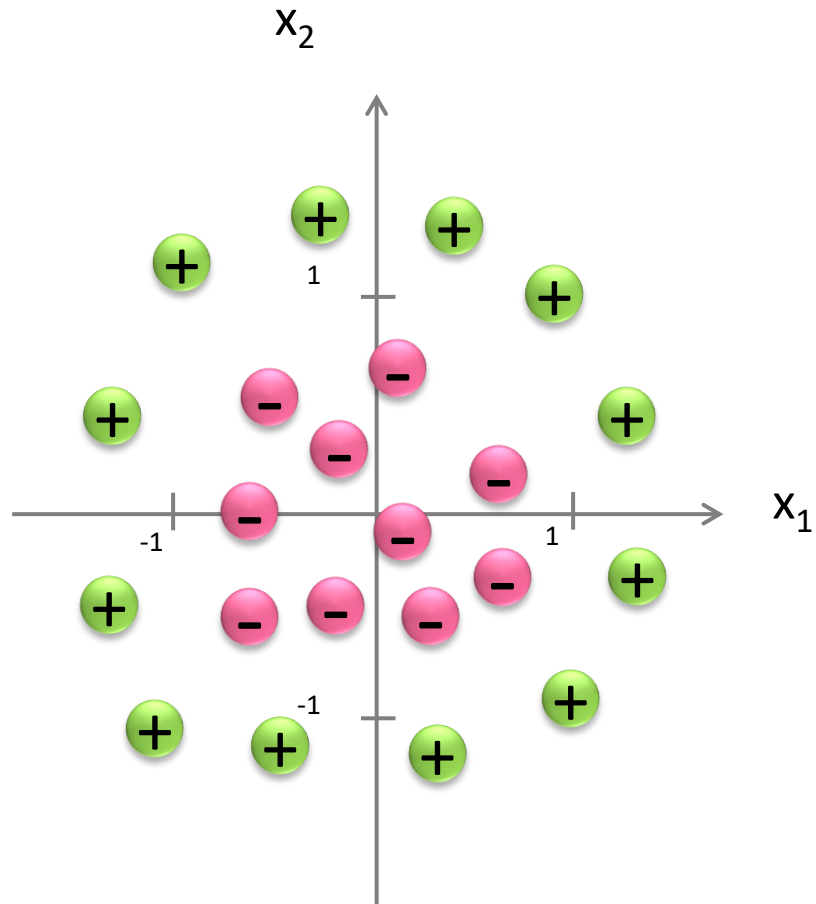
If the number of input **features is 3**, then the hyperplane becomes a **two-dimensional plane**.

It becomes **difficult** to imagine when the number of **features exceeds 3**.

Linear Decision Boundary with Hyperplane



Non-Linear Decision Boundary with Hyperplane



Support Vector Machines - SVM

A **Support Vector Machine (SVM)** is a very powerful Machine Learning model, capable of performing **linear or nonlinear classification, regression**, and even **outlier detection**.

SVM is built on the basis of statistical learning methods and the **structural risk minimization** principle instead of empirical risk minimization.

The fundamental idea of the SVM technique is to take advantage of a linear or nonlinear model to **map the target input data into a higher dimensional** characteristic space

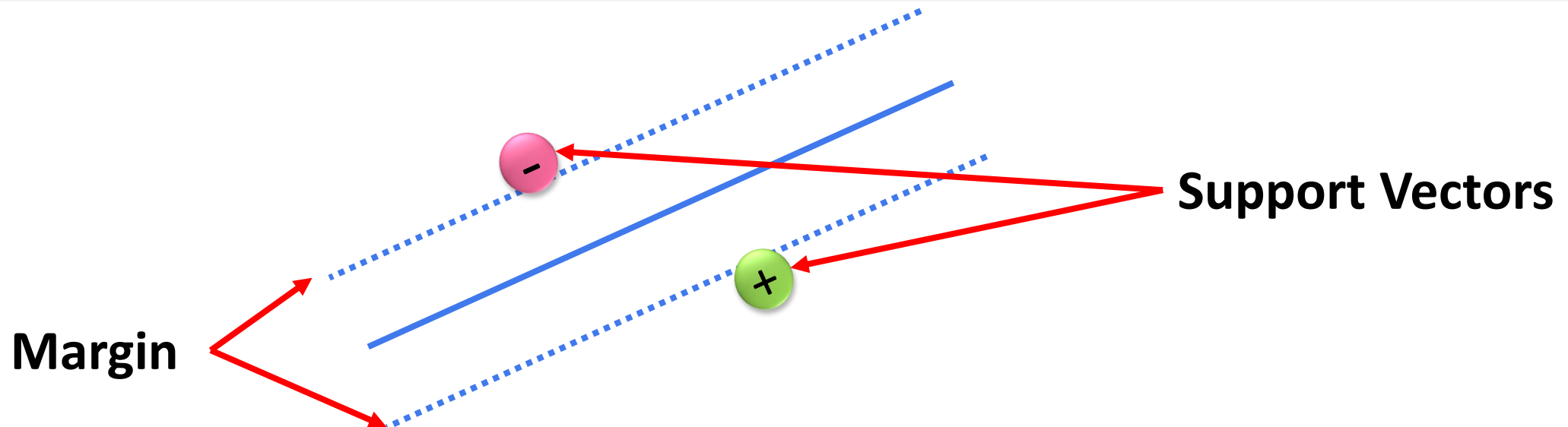
Kernel Trick: Transforming data into a higher-dimensional space to make it linearly separable. *So that the primary problem can be solved in the new space.*

Support Vectors

The main objective in **SVM** is to build the classifier whose decision boundary is furthest away from any data point.

The separating hyper-plane in terms of the data points that are closest to the boundary. And these points are called **support vectors**.

Margin is the distance between the **left hyperplane** and **right hyperplane**.



Support Vectors

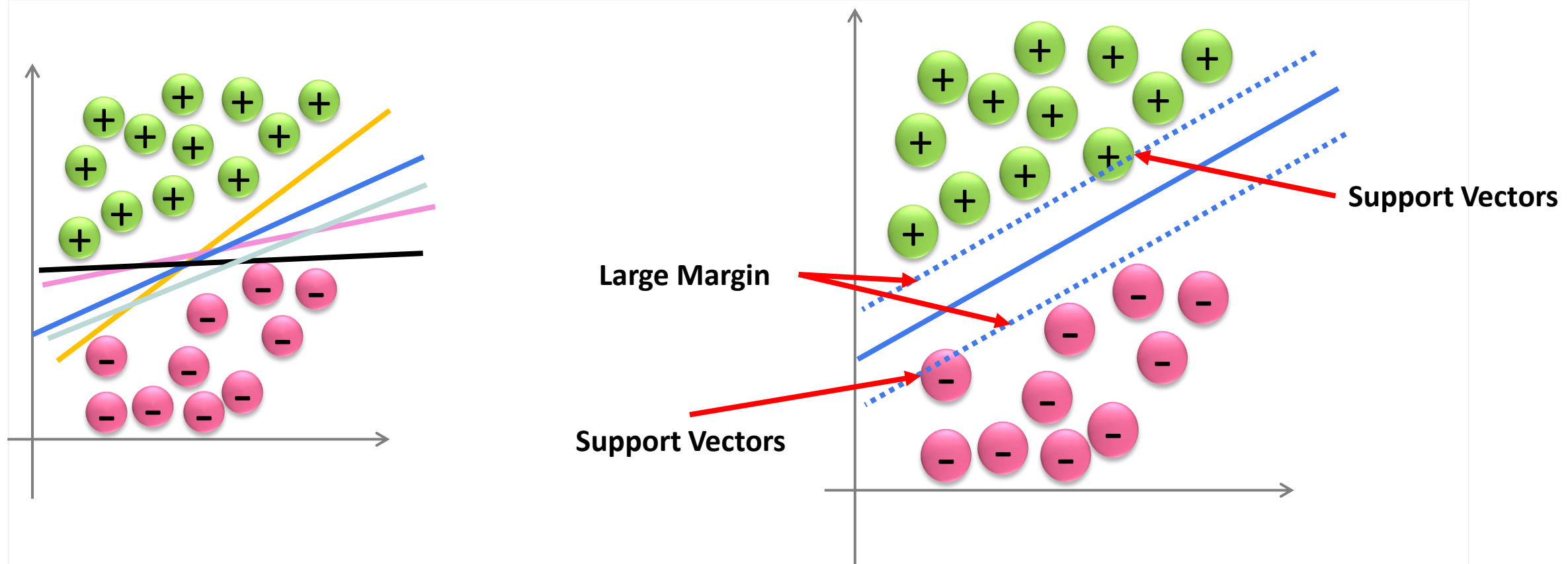
Support vectors are data points that are closer to the hyperplane and **influence** the **position** and **orientation** of the hyperplane.

Using these **support vectors**, we maximize the **margin** of the **classifier**.

Deleting the **support vectors** will **change the position of the hyperplane**.

Support Vectors are the points that help us build our SVM.

SVM for Binary Classification - Support Vectors



We can refer **SVM** classifier as fitting the widest possible street (represented by the parallel dashed lines) between the classes. This is called **large margin** classification.

SVM for Binary Classification - Large Margin

In **Logistic Regression**, we take the output of the linear function and squash the value within the range of $[0,1]$ using the sigmoid function.

If the squashed value is **greater than** a threshold value(**0.5**) we assign it a **label 1**, else we assign it a **label 0**.

In **SVM**, we take the output of the linear function and if that output is **greater than 1**, we identify it with one class and if the **output is -1**, we identify it with another class.

Since the threshold values are changed to **1 and -1 in SVM**, we obtain this reinforcement range of values($[-1,1]$) that acts as a **margin**.

Decision Boundary & Margin

How can we find the optimal hyperplane(solid blue line) ?

Maximize the margin(distance) between two hyperplanes then divide by 2 we get the decision boundary.

How to maximize the margin?

For 2 dimensions, the equation for hyper line is $w^T \cdot x + b = 0$ which is same as $w^T \cdot x = 0$

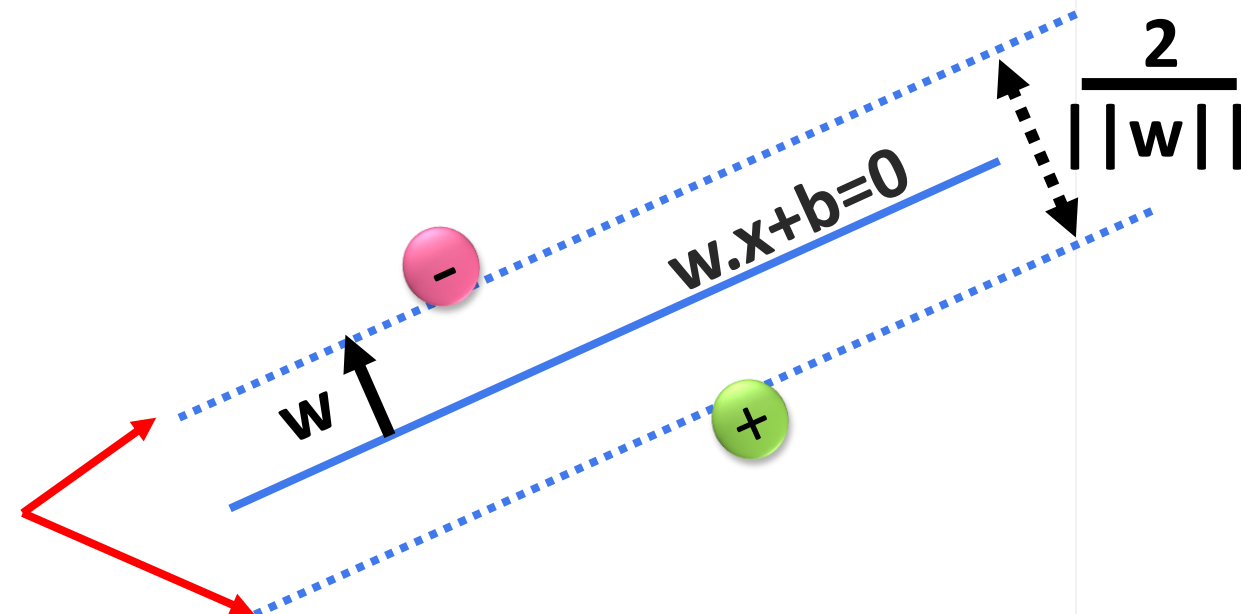
For each vector x_i either:

$w^T \cdot x_i + b \geq 1$ for x_i having the class 1

$w^T \cdot x_i + b \leq -1$ for x_i having the class -1

$$w \cdot x + b = 0$$

Margin



Decision Boundary & Margin

Case1:

If $w \cdot x + b = 1$ then we get (+)class hyperplane
for all positive(x) points satisfy this rule ($w \cdot x + b \geq 1$)

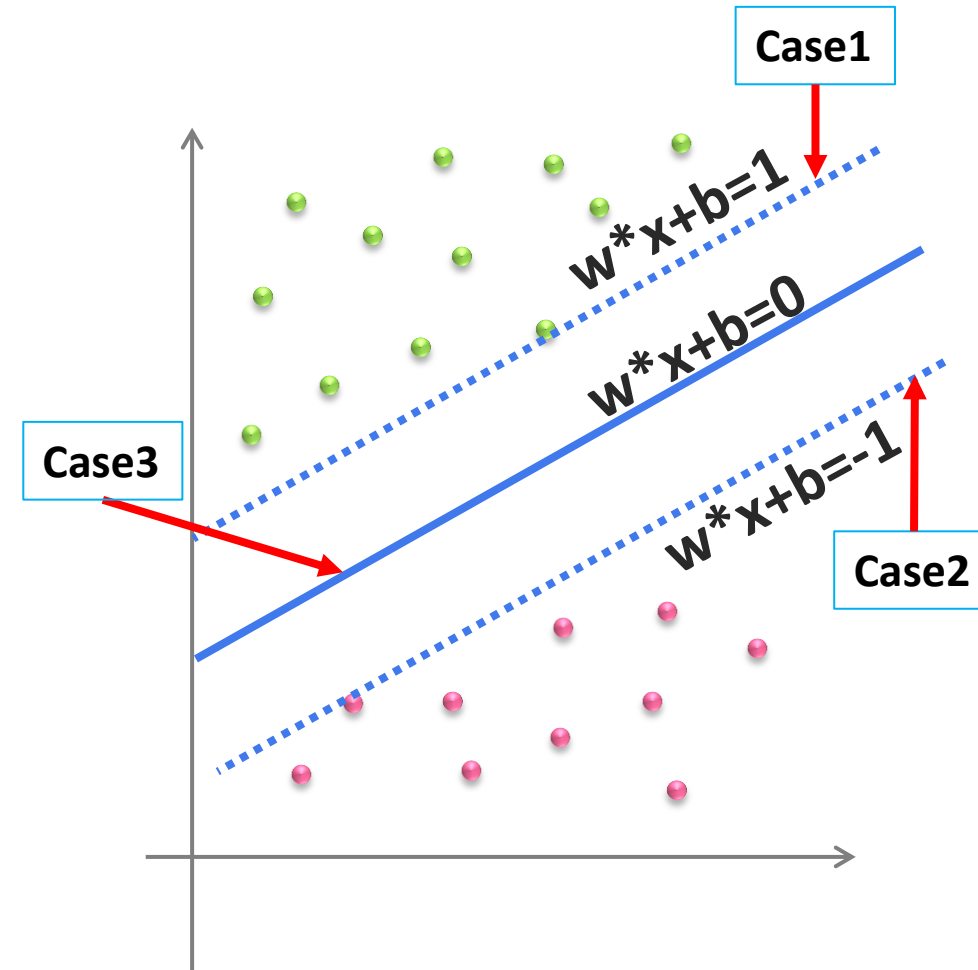
Case2:

If $w \cdot x + b = -1$ then we get (-)class hyperplane
for all negative(x) points satisfy rule ($w \cdot x + b \leq -1$)

Case3:

If $w \cdot x + b = 0$ then we get the decision boundary

The blue solid line



Maximize Margin

First Margin

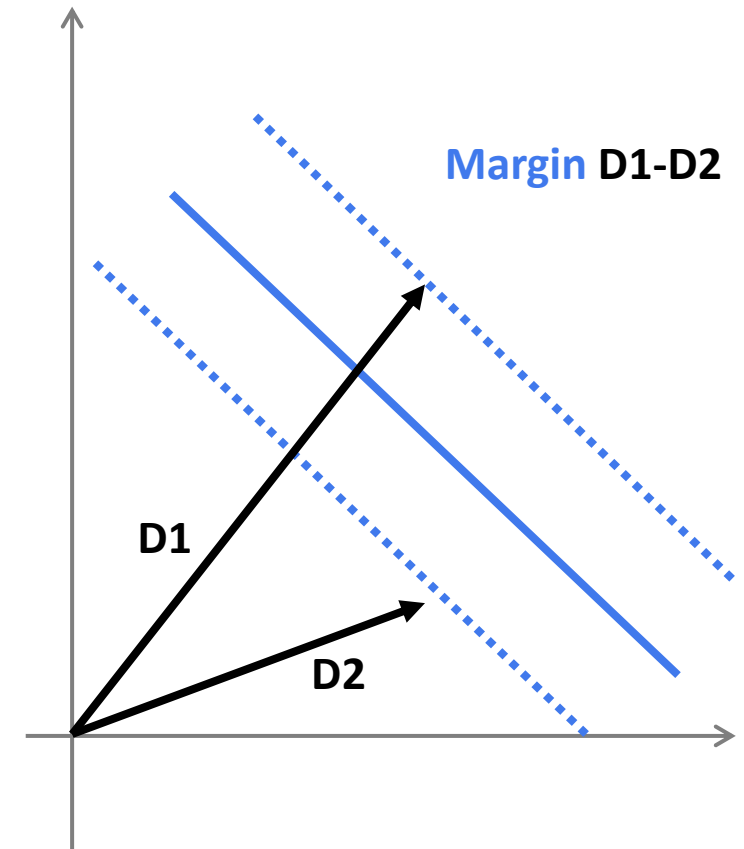
$$D1 = w^T \cdot x + b = 1 \text{ then } w^T \cdot x + b - 1 = 0$$

Second Margin

$$D2 = w^T \cdot x + b = -1 \text{ then } w^T \cdot x + b + 1 = 0$$

$$(w^T \cdot x + b - 1) - (w^T \cdot x + b + 1) = 2 / ||w||$$

To maximize the margin you need $\frac{1}{2} ||w||^2$

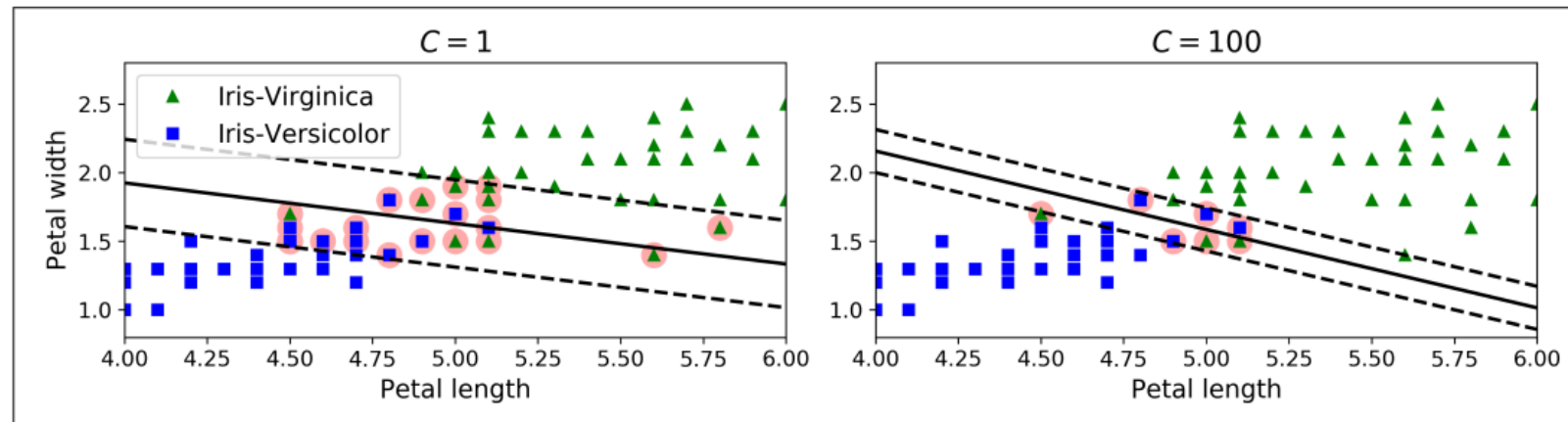


Soft Margin

A good balance between keeping the street as large as possible and limiting the margin violations
(i.e., instances that **end up in the middle** of the street or even on the wrong side).

This is called **soft margin** classification.

In Scikit-Learn's SVM classes, this balance can be controlled using the **C hyperparameter**: a smaller C value leads to a wider street but more margin violations.



Predict a Class - SVM for Binary Classification

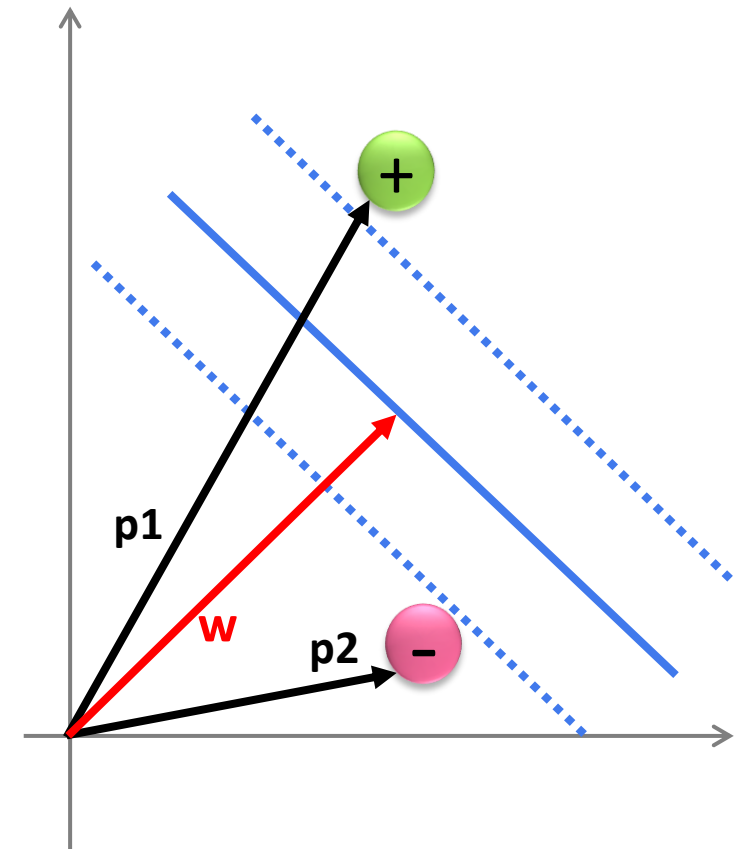
Now we give a unknown point and we want to predict whether it belongs to positive class or negative class.

First Point

$p1 = w^T \cdot x + b > 0$ then we get (+)

Second Point

$p2 = w^T \cdot x + b < 0$ then we get (-) class



SVM For Multiclass Classification

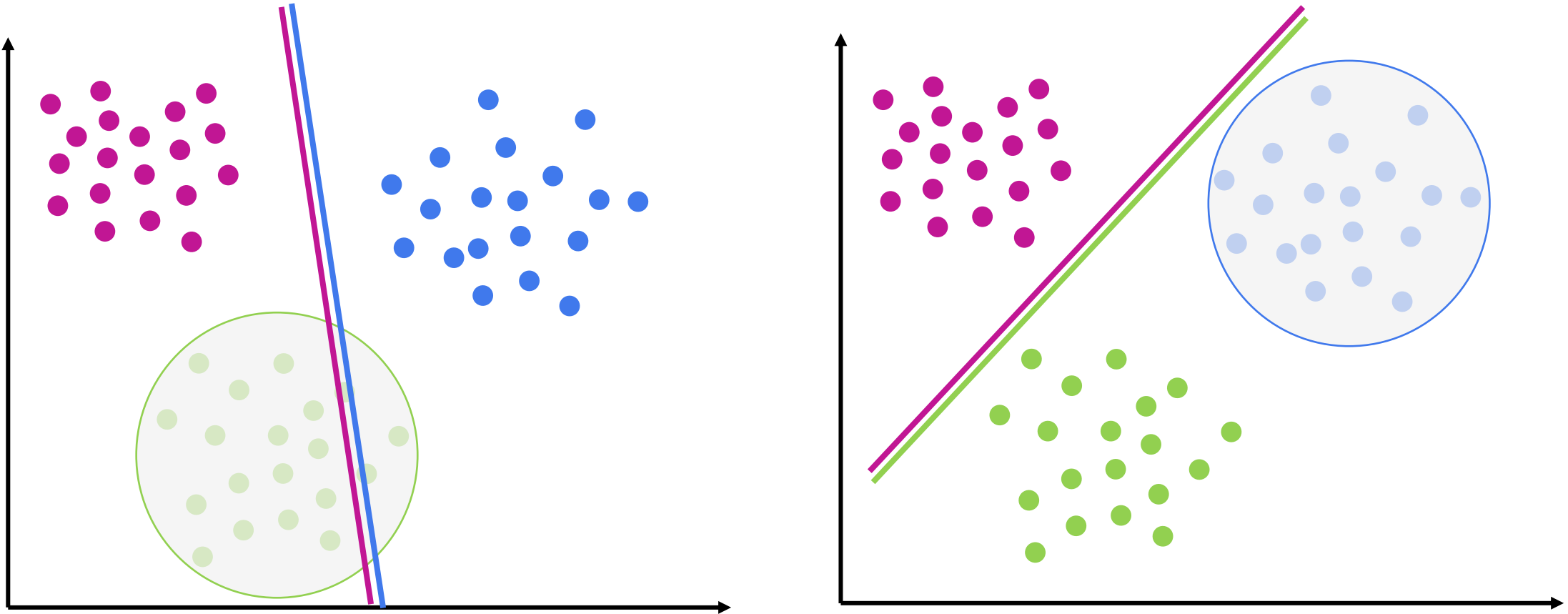
SVM is originally designed for binary classification, but it can be extended to handle **multi-class problems**.

Common approaches include **One-vs-One** (OvO) and **One-vs-All** (OvA)

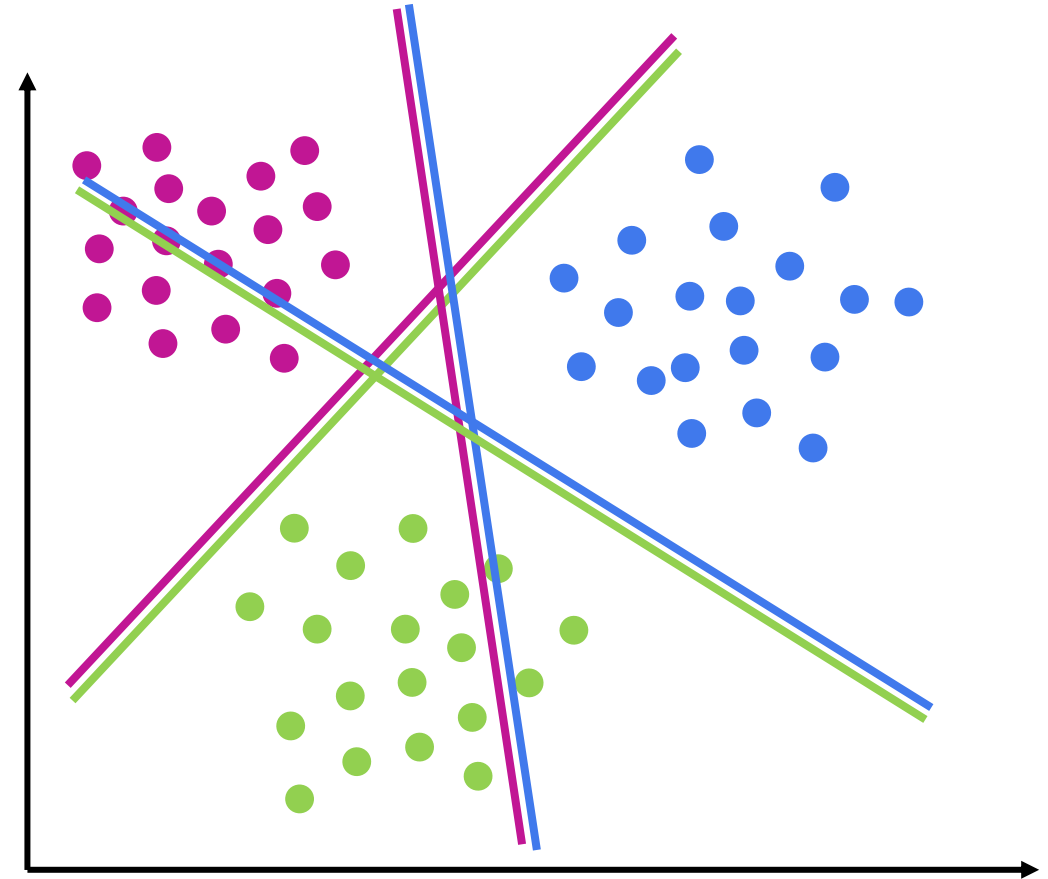
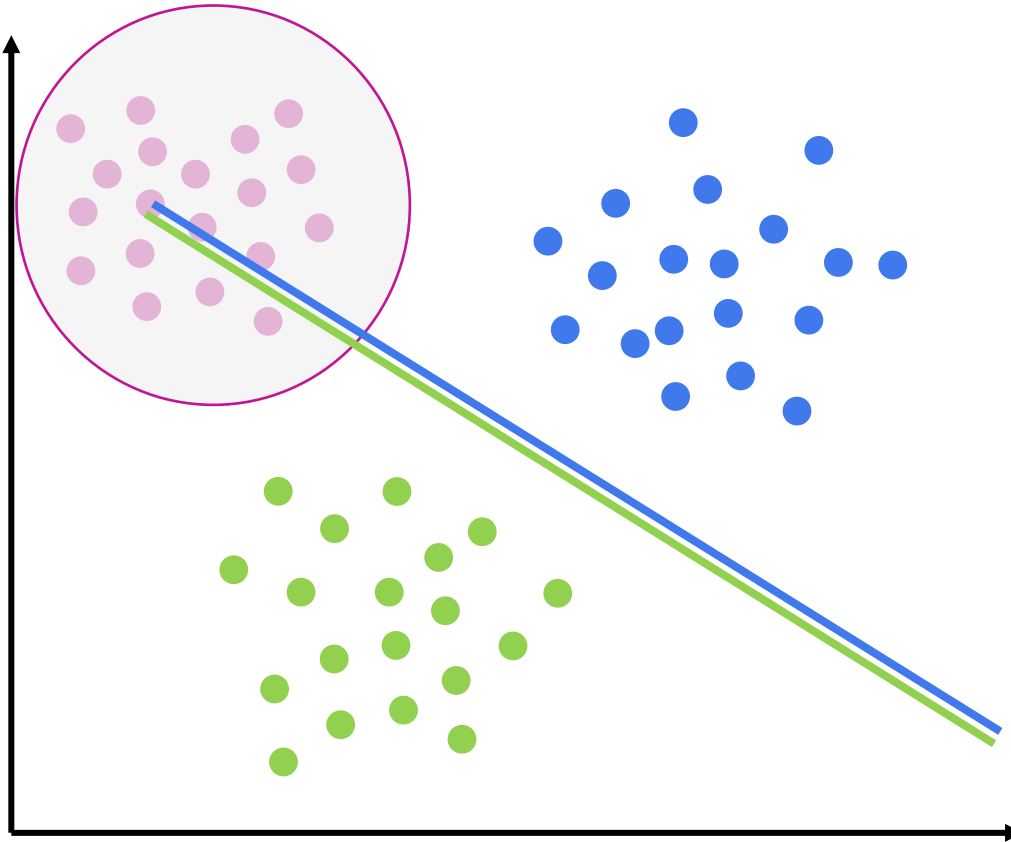
The **One-vs-One** approach breaks down the multiclass problem into multiple binary classification problems. A binary classifier per **each pair of classes**.

In One-vs-all binary classifiers are trained for **each class against all others**, respectively.

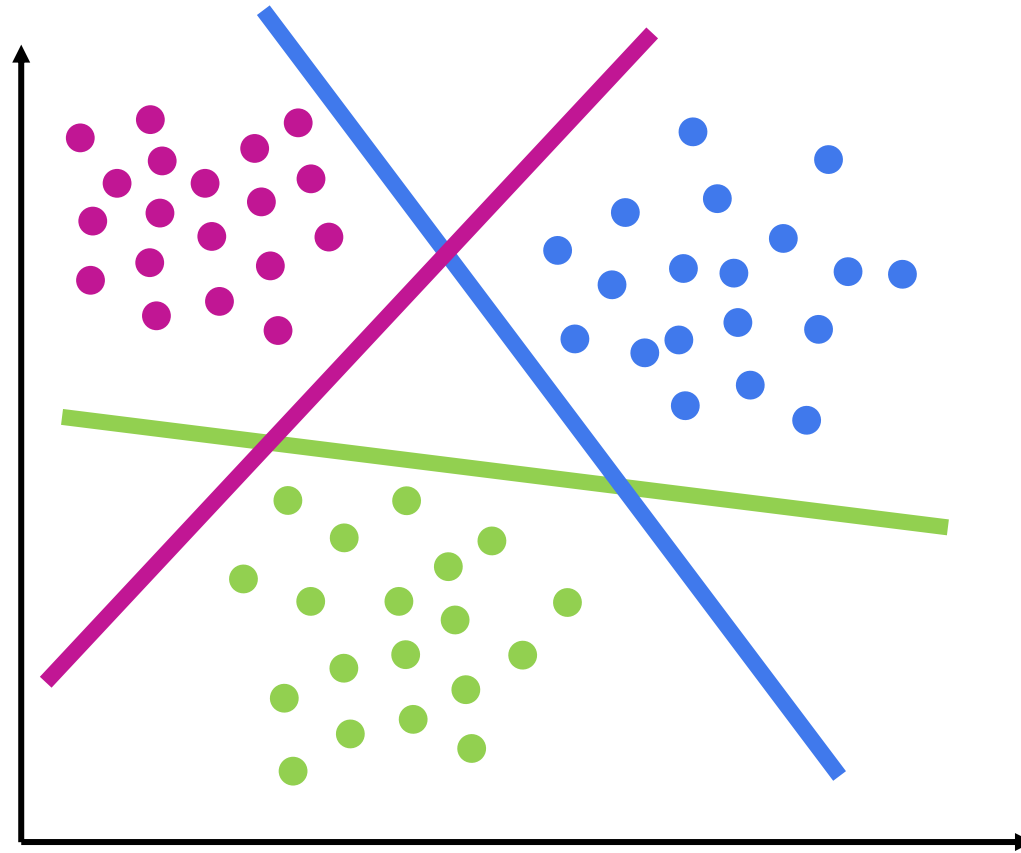
SVM For Multiclass Classification – One-vs-One



SVM For Multiclass Classification – One-vs-One

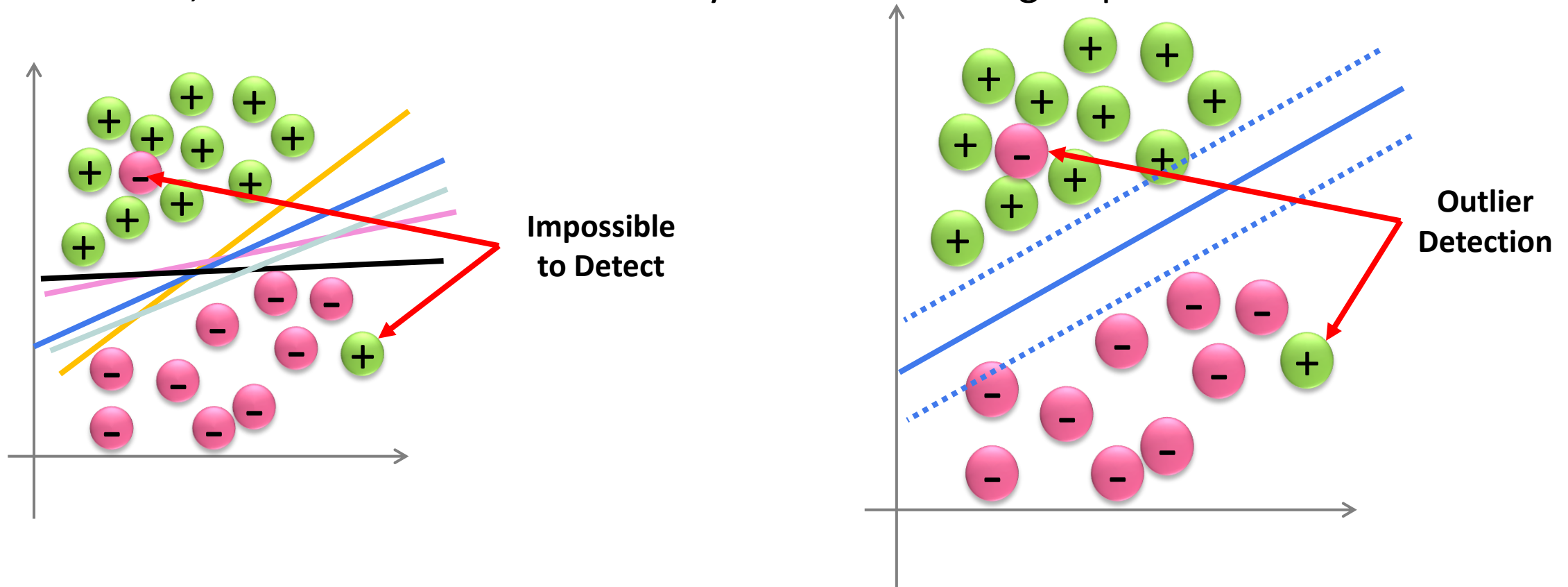


SVM For Multiclass Classification – One-vs-All



Outlier Detection

- An **outlier** is a value in a data set that is very different from the other values.
- That is, **outliers** are values unusually far from a class group or cluster.



The **SVM** classifier is quite sensitive to outliers.

Advantages and Disadvantages

Advantages:

1. Effective in high-dimensional spaces
2. Memory-efficient
3. Adapted to many different functions due to kernel trick, and
4. Good generalization if the right hyperparameters are chosen

Disadvantages:

1. Computationally intensive for large datasets
2. Sensitive to hyperparameter tuning, and
3. Challenging to interpret the model

Applications of SVM

SVM's are powerful and flexible supervised machine learning algorithms that are used both for classification and regression.

It is also one of the most efficient algorithms used for smaller datasets. There are many applications of SVM.

1. Facial Expression Classification
2. Text Classification
3. Speech Recognition
4. Medical Imaging (Disease Detection)

END
